RAPPORT

Listes Chaînées en Langage C

Remplissage & Affichage — Structures de Données

Module :	Algorithmique et Programmation Procédurale
Niveau :	BTS-DAI — Semestre 2
Établissement :	Lycée My Mohamed Ben Abdellah
Enseignant :	Pr. D. MOUJAHID (Informatique)
Année :	2025 / 2026

TABLE DE MATIERS

1. Introduction aux Listes Chaînées 2
2. Représentation en Mémoire 2
3. Déclaration de la Structure en C 3
4. Remplissage de la Liste 3
4.1 Insertion en tête (prepend) 3
4.2 Insertion en queue (append) 4
4.3 Remplissage interactif (saisie clavier) 4
5. Affichage de la Liste 4
5.2 Affichage récursif 5
5.3 Affichage en ordre inverse (récursif) 5
6. Autres Opérations Courantes 5
6.1 Longueur de la liste 5
6.2 Recherche d'un élément 6
6.3 Suppression d'un élément 6
6.4 Libération de la mémoire 7
7. Programme Complet — Exemple Exécutable 8
8. Types de Listes Chaînées 9
9. Complexité des Opérations 9
10. Conclusion 9

1. Introduction aux Listes Chaînées

Une liste chaînée est une structure de données **dynamique** constituée d'un ensemble de noeuds reliés entre eux par des pointeurs. Contrairement aux tableaux dont la taille est fixée à la compilation, les listes chaînées permettent d'allouer et de libérer de la mémoire à l'exécution, offrant ainsi une grande flexibilité pour gérer des collections de taille variable.

✓ Avantages	✗ Inconvénients
• Insertion/suppression en O(1) en tête • Taille dynamique • Pas de gaspillage mémoire prédéfini	• Accès séquentiel uniquement (pas d'accès direct) • Surcoût mémoire dû aux pointeurs

2. Représentation en Mémoire

Chaque noeud d'une liste chaînée simple contient deux champs :

- **data** (ou valeur) : la donnée stockée dans le noeud.
- **next** (ou suivant) : un pointeur vers le noeud suivant (*NULL* si c'est le dernier noeud).

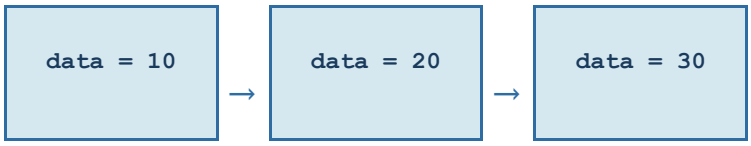


Figure 1 : Liste chaînée 10 → 20 → 30 → NULL

3. Déclaration de la Structure en C

En C, un noeud de liste chaînée se déclare à l'aide d'une structure **auto-référentielle** :

```
/* Déclaration d'un noeud */

typedef struct Noeud {

int data; /* donnée stockée */

struct Noeud *next; /* pointeur vers suivant */ }

Noeud;


/* Pointeur de tête de liste */

Noeud *tete = NULL; /* liste vide au départ */
```

Le mot-clé **typedef** permet de définir un alias **Noeud** pour éviter de répéter *struct Noeud* partout dans le code. La liste est représentée par un simple pointeur **tete** qui pointe sur le premier noeud (*NULL* si la liste est vide).

4. Remplissage de la Liste

On peut insérer des noeuds en tête ou en queue. L'insertion en tête est la plus simple et s'effectue en **O(1)** ; l'insertion en queue nécessite de parcourir la liste entière.

4.1 Insertion en tête (prepend)

```
Noeud* inserer_tete(Noeud *tete, int valeur) {
```

```
Noeud *nouveau = (Noeud*) malloc(sizeof(Noeud));

if (nouveau == NULL) {

fprintf(stderr, "Erreur d'allocation\n");

exit(EXIT_FAILURE);

}

nouveau->data = valeur;

nouveau->next = tete; /* l'ancienne tête devient 2e */

return nouveau; /* nouveau pointeur de tête */

}
```

4.2 Insertion en queue (append)

```
Noeud* inserer_queue(Noeud *tete, int valeur) {

Noeud *nouveau = (Noeud*) malloc(sizeof(Noeud));

if (nouveau == NULL) { exit(EXIT_FAILURE); }

nouveau->data = valeur;

nouveau->next = NULL;

if (tete == NULL) return nouveau; /* liste vide */

Noeud *courant = tete;

while (courant->next != NULL) /* aller à la fin */

courant = courant->next;

courant->next = nouveau;

return tete; }
```

4.3 Remplissage interactif (saisie clavier)

```
Noeud* remplir_liste(int n) { Noeud *tete = NULL;

int val;

printf("Saisir %d entiers :\n", n);

for (int i = 0; i < n; i++) {

printf(" Element [%d] : ", i);

scanf("%d", &val);

tete = inserer_queue(tete, val);
```

```
}

return tete;

}
```

5. Affichage de la Liste

L'affichage d'une liste chaînée consiste à parcourir chaque noeud depuis la tête jusqu'au pointeur NULL, en affichant la donnée de chaque noeud. **5.1 Affichage itératif**

```
void afficher_liste(Noeud *tete) {
if (tete == NULL) { printf("[Liste vide]\n"); return; }
Noeud *courant = tete;
printf("Liste : ");
while (courant != NULL) {
printf("%d", courant->data);
if (courant->next != NULL) printf(" -> ");
courant = courant->next;
}
printf(" -> NULL\n");
}
```

5.2 Affichage récursif

```
void afficher_rec(Noeud *courant) {
if (courant == NULL) { printf("NULL\n"); return; }
printf("%d -> ", courant->data);
afficher_rec(courant->next); }
```

5.3 Affichage en ordre inverse (récursif)

```
void afficher_inverse(Noeud *courant) {
if (courant == NULL) return;
afficher_inverse(courant->next);
printf("%d ", courant->data);
}
```

6. Autres Opérations Courantes

6.1 Longueur de la liste

```
int longueur_liste(Noeud *tete) {
int count = 0;
while (tete != NULL) { count++; tete = tete->next; }
return count; }
```

6.2 Recherche d'un élément

```
int rechercher(Noeud *tete, int val) {

while (tete != NULL) {

if (tete->data == val) return 1;

tete = tete->next;

}

return 0;

}
```

6.3 Suppression d'un élément

```


Noeud* supprimer(Noeud *tete, int val) {

if (tete == NULL) return NULL;

if (tete->data == val) {

Noeud *tmp = tete->next;

free(tete); return tmp; } Noeud *prec =

tete, *courant = tete->next;

while (courant != NULL && courant->data != val) {

prec = courant; courant = courant->next;

}

if (courant != NULL) {

prec->next = courant->next;

free(courant);

}

return tete;


}
```

6.4 Libération de la mémoire

```
void liberer_liste(Noeud *tete) {

Noeud *tmp;

while (tete != NULL) {

tmp = tete->next;

free(tete);

tete = tmp;

}

}
```

7. Programme Complet — Exemple Exécutable

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Noeud { int data; struct Noeud *next; } Noeud;

/* --- Prototypes --- */
Noeud* inserer_tete (Noeud *tete, int v);
Noeud* inserer_queue(Noeud *tete, int v);
void afficher_liste(Noeud *tete);
int longueur_liste(Noeud *tete);
int rechercher (Noeud *tete, int val);
Noeud* supprimer (Noeud *tete, int val);
void liberer_liste (Noeud *tete);

int main(void) {
Noeud *liste = NULL;
for (int i = 1; i <= 5; i++)
liste = inserer_queue(liste, i * 10);
printf("Longueur : %d\n", longueur_liste(liste));
afficher_liste(liste);
printf("30 present ? %s\n",
rechercher(liste, 30) ? "OUI" : "NON");
liste = supprimer(liste, 30);
printf("Apres suppression de 30 :\n");
afficher_liste(liste);
liberer_liste(liste);
return 0;
}
```

Compilation :

```
gcc -o liste liste.c && ./liste
```

Sortie attendue :

```
Longueur : 5 Liste : 10 -> 20 -> 30 -> 40
-> 50 -> NULL
30 present ? OUI Apres
suppression de 30 :
Liste : 10 -> 20 -> 40 -> 50 -> NULL
```

8. Types de Listes Chaînées

Type	Description	Structure next/prev
Simple (SLL)	Chaque noeud pointe vers le suivant.	next → NULL
Double (DLL)	Chaque noeud possède deux pointeurs : précédent et suivant.	prev noeud → next
Circulaire (CLL)	Le dernier noeud pointe sur le premier.	next du dernier = tete
Circulaire double	Combinaison DLL + circulaire.	prev/next bouclent

Structure d'un noeud doublement chaîné :

```
typedef struct NoeudD {

int data;

struct NoeudD *prev; /* pointeur vers le précédent
*/

struct NoeudD *next; /* pointeur vers le suivant */

} NoeudD;
```

9. Complexité des Opérations

Opération	Liste chaînée simple	Tableau
Accès par indice	O(n)	O(1)
Insertion en tête	O(1)	O(n)
Insertion en queue	O(n)*	O(1) amorti
Insertion au milieu	O(n)	O(n)
Suppression en tête	O(1)	O(n)
Suppression au milieu	O(n)	O(n)
Recherche séquentielle	O(n)	O(n)

** O(1) si on maintient un pointeur queue en plus du pointeur tête.*

10. Conclusion

Les listes chaînées constituent l'une des structures de données fondamentales en programmation. Leur maîtrise est indispensable pour tout développeur, car elles servent de base à de nombreuses structures plus avancées (piles, files, arbres, graphes).

Points clés à retenir :

- Toujours initialiser le pointeur tête à **NULL**.
- Utiliser **malloc()** pour créer chaque noeud et vérifier le retour (différent de NULL).
- Toujours libérer la mémoire avec **free()** après utilisation.
- Préférer l'insertion en tête O(1) quand l'ordre n'importe pas.
- Garder un pointeur de queue pour l'insertion en O(1) en fin de liste.

Conseil : Dessinez toujours un schéma de la liste avant d'écrire le code. Visualiser les pointeurs évite la majorité des erreurs (segmentation fault, fuites mémoire).